SWE 4101

Object Oriented Concepts I

Prepared By:
Samia Nawsheen
Junior Lecturer, CSE

Objectives

01 Two questions about design

02 What is coupling

03 What is cohesion

04 How OOP concepts connect

05 Classifying real examples

06 An important nuance

Two Questions About Design

- How much does one part of a program depend on another? — that's coupling
- How closely do the responsibilities inside one class actually belong together? — that's cohesion
- The one sentence to remember: low coupling + high cohesion = generally good design

A Quick Definition: Composition

- Composition means building a class using objects of other classes, held as fields
- Often called a “has-a” relationship
- A Car has an Engine, rather than a Car is an Engine

What is Coupling

- Coupling measures how much one class depends on another
- High coupling: change one class, and you're often forced to change another
- Low coupling: classes can change independently of each other

A Simple Coupling Example

```
class Order {
    private PaymentService paymentService;

    public void placeOrder() {
        paymentService.processPayment();
    }
}
```

- Order depends on PaymentService, but only through one method call
- If PaymentService changes how it works internally, Order never has to know
- This is what low coupling looks like in practice

Question for the class

> *If we completely changed how payment works internally, would Order care?*

Working it through

- As long as processPayment() still exists and does its job, Order doesn't care at all
- Order depends on the behavior
- Not on how that behavior is implemented

The Tightly Coupled Version

```
class Order {
    private String cardNumber;
    private String cvv;

    public void placeOrder() {
        // validate card, contact bank, charge card...
        // all written directly inside Order
    }
}
```

- Now Order knows every detail of how payment actually works
- Any change to payment logic means editing Order directly
- Order and “how payment works” can no longer change independently

An Important Nuance

- Coupling isn't automatically bad
- Classes have to depend on each other to get anything done
- The goal isn't zero coupling
- It's avoiding unnecessary, rigid dependencies
- Ask: does this class depend on what another class does, or exactly how it does it?

What is Cohesion

- Cohesion measures how closely the responsibilities inside a class belong together
- High cohesion: everything in the class serves one clear purpose
- Low cohesion: the class mixes together jobs that don't really relate

A Simple Cohesion Example

```
class Student {
    void calculateCGPA() { }
    void registerCourse() { }
    void printResult() { }
}
```

- calculateCGPA and registerCourse are clearly about being a student
- printResult is a formatting/output concern, not really “student” behavior
- Not every method that touches Student's data has to live inside Student

Question for the class

Do all three of these responsibilities really belong together in one class?

Working it through

- calculateCGPA and registerCourse both describe things a student does
- printResult is about displaying something
- It is a separate concern entirely
- Separating display from behavior will improve cohesion

Splitting Out the Unrelated Part

```
class Student {
    void calculateCGPA() { }
    void registerCourse() { }
}

class ResultPrinter {
    void printResult() { }
}
```

- Student now only does student things
- ResultPrinter has exactly one job, and its name says so
- Neither class changes because of the other's concerns

Connecting to What You Already Know

OOP Concept	Connection
Encapsulation	Bundles related data and behavior together, which supports high cohesion
Abstraction	Hides implementation details, so code depends on what something does, not how
Interfaces	Depending on a contract instead of a concrete class directly lowers coupling
Inheritance	A subclass often depends on its parent's internals, which can increase coupling
Composition	A class holds other objects as fields, instead of containing their logic itself

Why Interfaces Specifically Help

- Interfaces already help us attain low coupling
- One reason interfaces are useful is reducing how strongly one class depends on a particular implementation
- PayrollRun depending on LedgerSink instead of ConsoleSink was already a coupling decision

Now You Try

- For each example: is it loosely or tightly coupled? Highly or lowly cohesive?
- Then decide – what, if anything, would you change?
- There's often no single right answer – the reasoning matters more than the label

Example 1: Restaurant

```
class Restaurant {
    void takeOrder() { }
    void cookFood() { }
    void calculateSalary() { }
    void generateEmployeeReport() { }
}
```

- Which of these responsibilities are actually about running a restaurant?
- Which ones sound like a completely different job?
- What would you split this into?

Working it through

- takeOrder and cookFood are core restaurant responsibilities
- calculateSalary and generateEmployeeReport are HR/admin concerns, unrelated to running a kitchen
- Split into Restaurant and a separate employee-administration class

Example 2: Order, Built From Other Classes

```
class Order {
    Payment payment;
    Delivery delivery;
    Customer customer;
}
```

- Does Order need to directly create and control all three of these?
- What changes if Order only depends on what each one promises to do, not how?
- Is this coupling necessary, or could it be loosened?

Working it through

- Order composing Payment, Delivery, and Customer is reasonable; an order genuinely involves all three
- The real question is whether Order controls their internals, or just calls their public behavior
- Depending on abstractions here would loosen this further, without removing the relationship entirely

Practice exercise

1. Take a class from your own lab work with more than two responsibilities
2. Classify it: does everything genuinely belong together, or is it coincidental?
3. Find one place your code depends on another class's exact implementation, not just its behavior
4. Rewrite that spot so it depends only on what the other class promises to do

Thank You